1.7. MySQL8新特性

对于 MySQL 5.7 版本，其将于 2023年 10月31日 停止支持。后续官方将不再进行后续的代码维护。

MySQL 8.0 全内存访问可以轻易跑到 200W QPS，I/O 极端高负载场景跑到 16W QPS，除此之外 MySQL 8还新增了很多功能，那么我们来一起看一下。

1.7.1. 账户与安全

1.7.1.1. 用户创建和授权

到了MySQL8中，用户创建与授权语句必须是分开执行，之前版本是可以一起执行。

```
mysql> \s
--------------
mysql  Ver 8.0.29 for Win64 on x86_64 (MySQL Community Server - GPL)

Connection id:          8
Current database:
Current user:           root@localhost
SSL:                    Cipher in use is TLS_AES_256_GCM_SHA384
Using delimiter:        ;
Server version:         8.0.29 MySQL Community Server - GPL
Protocol version:       10
Connection:             localhost via TCP/IP
Server characterset:    utf8mb4
Db     characterset:    utf8mb4
Client characterset:    gbk
Conn.  characterset:    gbk
TCP port:               3306
Binary data as:         Hexadecimal
Uptime:                 2 min 18 sec

Threads: 2  Questions: 5  Slow queries: 0  Opens: 117  Flush tables: 3  Open tables: 36  Queries p
r second avg: 0.036
--------------
```

MySQL8的版本

```
grant all privileges on *.* to 'lijin'@'%' identified by 'Lijin@2022';
```

```
mysql> grant all privileges on *.* to 'lijin'@'%' identified by 'Lijin@2022';
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server versi
on for the right syntax to use near 'identified by 'Lijin@2022'' at line 1
```

```
create user 'lijin'@'%' identified by 'Lijin@2022';
grant all privileges on *.* to 'lijin'@'%';
```

```
mysql8 >create user 'lijin'@'%' identified by 'Lijin@2022';
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> grant all privileges on *.* to 'lijin'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> select host,user from mysql.user;
+-----------+------------------+
| host      | user             |
+-----------+------------------+
| %         | lijin            |
| localhost | mysql.infoschema |
| localhost | mysql.session    |
| localhost | mysql.sys        |
| localhost | root             |
+-----------+------------------+
5 rows in set (0.00 sec)
```

MySQL5.7的版本

```
grant all privileges on *.* to 'lijin'@'%' identified by 'Lijin@2022';
```

```
mysql> grant all privileges on *.* to 'lijin'@'%' identified by 'Lijin@2022';
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> select host,user from mysql.user;
+-----------+--------------+
| host      | user         |
+-----------+--------------+
| %         | lijin        |
| %         | root         |
| localhost | mysql.session |
| localhost | mysql.sys    |
| localhost | root         |
+-----------+--------------+
5 rows in set (0.00 sec)
```

1.7.1.2. 认证插件更新

MySQL 8.0中默认的身份认证插件是caching_sha2_password，替代了之前的mysql_native_password。

```
show variables like 'default_authentication%';
```

5.7版本

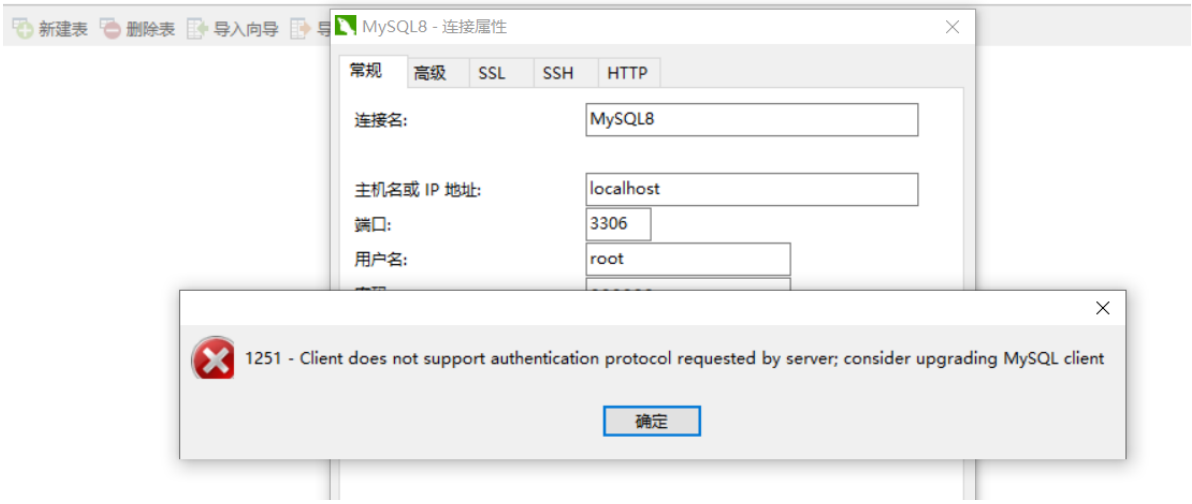
```
mysql> show variables like 'default_authentication%';
+-------------------------------+-----------------------+
| Variable_name                 | Value                 |
+-------------------------------+-----------------------+
| default_authentication_plugin | mysql_native_password |
+-------------------------------+-----------------------+
1 row in set (0.00 sec)
```

8版本

```
mysql> show variables like 'default_authentication%';
+-------------------------------+-----------------------+
| Variable_name                 | Value                 |
+-------------------------------+-----------------------+
| default_authentication_plugin | caching_sha2_password |
+-------------------------------+-----------------------+
```

```
select user, host,plugin from mysql.user;
```

这个带来的问题就是如果客户端没有更新，就连接不上！！



当然可以通过在MySQL的服务端找到my.cnf的文件，把相关参数进行修改（不过要MySQL重启后才能生效）

```
# Remove leading # to revert to previous value for default_authentication_plugin,
# this will increase compatibility with older clients. For background, see:
# https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html#sysvar_default_au
hentication_plugin
# default-authentication-plugin=mysql_native_password
```

把这段注释放开即可

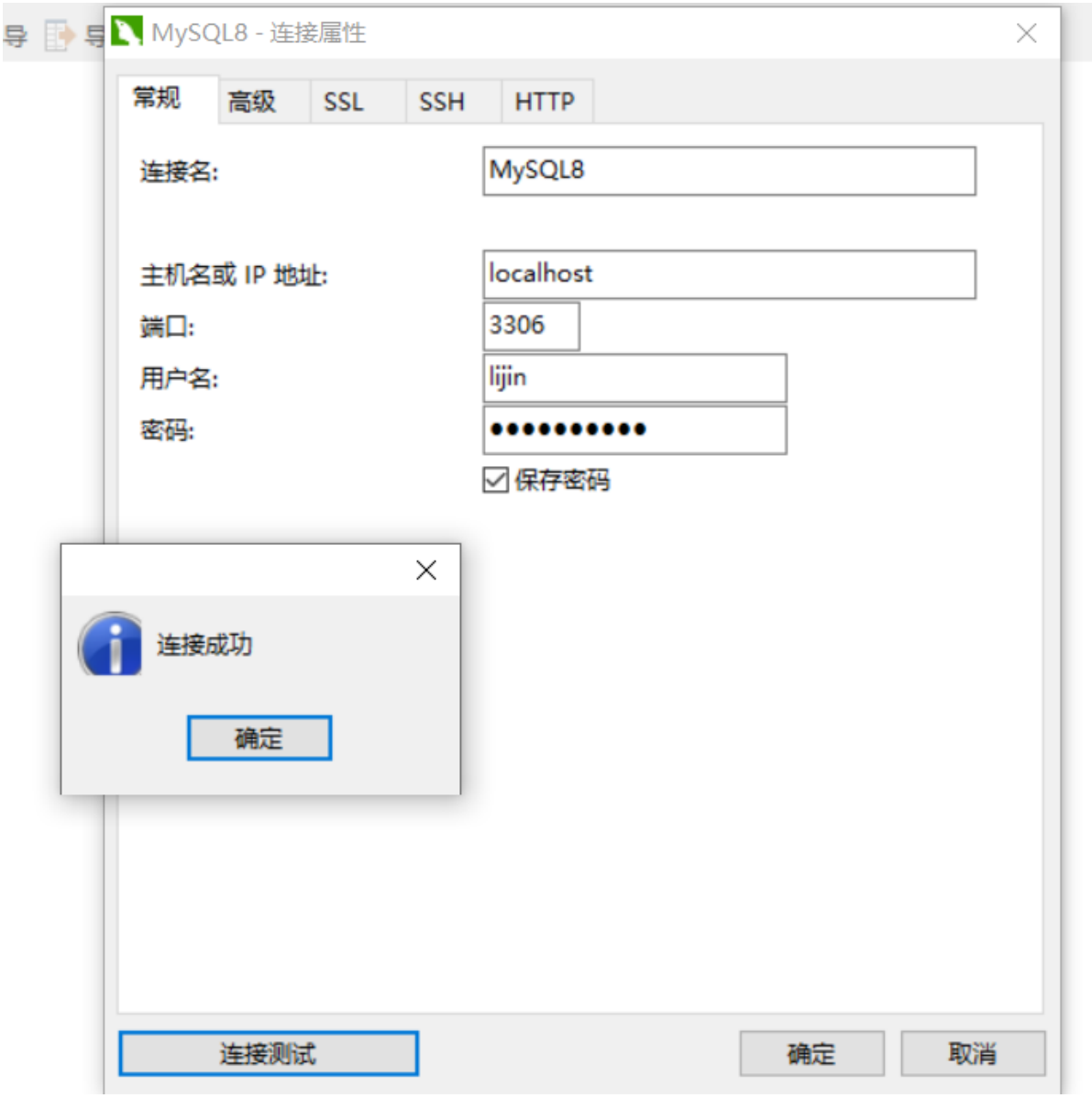
如果没办法重启服务，还有一种动态的方式：

```
alter user 'lijin'@'%' identified with mysql_native_password by 'Lijin@2022';
select host,user from mysql.user;
```

```
mysql> alter user 'lijin'@'%' identified with mysql_native_password by 'Lijin@2022';
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select user, host,plugin from mysql.user;
+------------------+-----------+-----------------------+
| user             | host      | plugin                |
+------------------+-----------+-----------------------+
| lijin            | %         | mysql_native_password |
| mysql.infoschema | localhost | caching_sha2_password |
| mysql.session    | localhost | caching_sha2_password |
| mysql.sys        | localhost | caching_sha2_password |
| root             | localhost | caching_sha2_password |
+------------------+-----------+-----------------------+
```

使用老的Navicat for MySQL也能访问

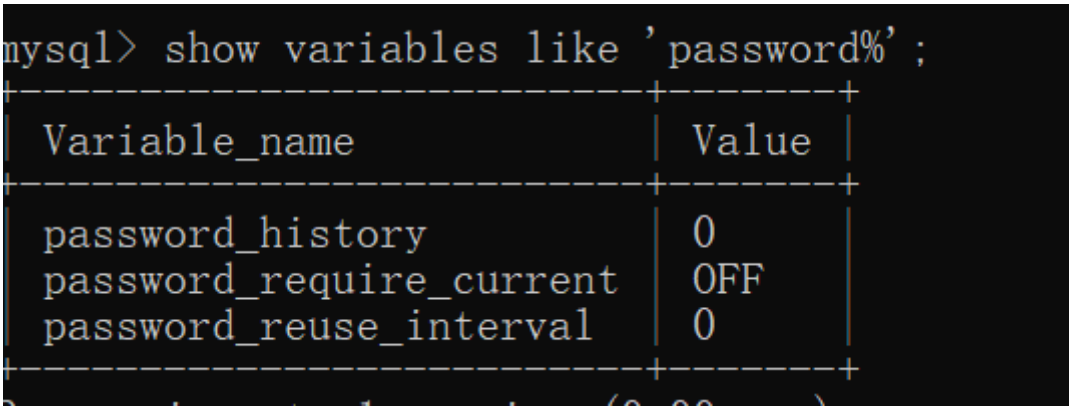


1.7.1.3. 密码管理

MySQL 8.0开始允许限制重复使用以前的密码（修改密码时）。

并且还加入了密码的修改管理功能

```
show variables like 'password%';
```



修改策略（全局级）

```
set persist password_history=3;          --修改密码不能和最近3次一致
```

```
mysql> set persist password_history=3;
Query OK, 0 rows affected (0.00 sec)

mysql> show variables like 'password%';
+--------------------------+-------+
| Variable_name            | Value |
+--------------------------+-------+
| password_history         | 3     |
| password_require_current | OFF   |
| password_reuse_interval  | 0     |
+--------------------------+-------+
```

修改策略（用户级）

```
alter user 'lijin'@'%' password history 3;
```

```
select user, host,Password_reuse_history from mysql.user;
```

```
mysql> alter user 'lijin'@'%' password history 3;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> select user, host,Password_reuse_history from mysql.user;
+------------------+-----------+-------------------------+
| user             | host      | Password_reuse_history  |
+------------------+-----------+-------------------------+
| lijin            | %         |                       3 |
| mysql.infoschema | localhost |                    NULL |
| mysql.session    | localhost |                    NULL |
| mysql.sys        | localhost |                    NULL |
| root             | localhost |                    NULL |
+------------------+-----------+-------------------------+
5 rows in set (0.00 sec)
```

使用重复密码修改用户密码(指定lijin用户)

```
alter user 'lijin'@'%' identified by 'Lijin@2022';
```

```
mysql> alter user 'lijin'@'%' identified by 'Lijin@2022';
ERROR 3638 (HY000): Cannot use these credentials for 'lijin@%' because they contradict the password history policy
mysql>
```

如果我们把全局的参数改为0，则对于root用户可以反复的修改密码

```
alter user 'root'@'localhost' identified by '789456';
```

```
mysql> set persist password_history=0;
Query OK, 0 rows affected (0.00 sec)

mysql> alter user 'root'@'localhost' identified by '789456';
Query OK, 0 rows affected (0.01 sec)

mysql> alter user 'root'@'localhost' identified by '789456';
Query OK, 0 rows affected (0.01 sec)

mysql> alter user 'root'@'localhost' identified by '789456';
Query OK, 0 rows affected (0.01 sec)

mysql> alter user 'root'@'localhost' identified by '789456';
Query OK, 0 rows affected (0.01 sec)

mysql> alter user 'root'@'localhost' identified by '789456';
Query OK, 0 rows affected (0.01 sec)

mysql> alter user 'root'@'localhost' identified by '789456';
Query OK, 0 rows affected (0.01 sec)
```

password_reuse_interval 则是按照天数来限定（不允许重复的）

password_require_current 是否需要校验旧密码（off 不校验、 on校验）(针对非root用户)

```
set persist password_require_current=on;
```

1.7.2. 索引增强

1.7.2.1. 隐藏索引

MySQL 8.0开始支持隐藏索引 (invisible index)，不可见索引.

隐藏索引不会被优化器使用，但仍然需要进行维护。
应用场景: 软删除、灰度发布。

软删除：就是我们在线上会经常删除和创建索引，如果是以前的版本，我们如果删除了索引，后面发现删错了，我又需要创建一个索引，这样做的话就非常影响性能。在MySQL8中我们可以这么操作，把一个索引变成隐藏索引（索引就不可用了，查询优化器也用不上），最后确定要进行删除这个索引我们才会进行删除索引操作。

灰度发布：也是类似的，我们想在线上进行一些测试，可以先创建一个隐藏索引，不会影响当前的生产环境，然后我们通过一些附加的测试，发现这个索引没问题，那么就直接把这个索引改成正式的索引，让线上环境生效。

使用案例（灰度发布）：

```
create table t1(i int,j int);  --创建一张t1表
create index i_idx on t1(i);  --创建一个正常索引
create index j_idx on t1(j) invisible;  --创建一个隐藏索引
```

```
mysql> create table t1(i int,j int);
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> create index i_idx on t1(i);
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
mysql> create index j_idx on t1(j) invisible;
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
show index from t1\G          --查看索引信息
```

```
mysql> show index from t1\G
*************************** 1. row ***************************
        Table: t1
   Non_unique: 1
     Key_name: i_idx
 Seq_in_index: 1
  Column_name: i
    Collation: A
  Cardinality: 0
     Sub_part: NULL
       Packed: NULL
         Null: YES
   Index_type: BTREE
      Comment:
Index_comment:
      Visible: YES
   Expression: NULL
*************************** 2. row ***************************
        Table: t1
   Non_unique: 1
     Key_name: j_idx
 Seq_in_index: 1
  Column_name: j
    Collation: A
  Cardinality: 0
     Sub_part: NULL
       Packed: NULL
         Null: YES
   Index_type: BTREE
      Comment:
Index_comment:
      Visible: NO
   Expression: NULL
2 rows in set (0.01 sec)
```

使用查询优化器看下：

```
explain select * from t1 where i=1;
explain select * from t1 where j=1;
```

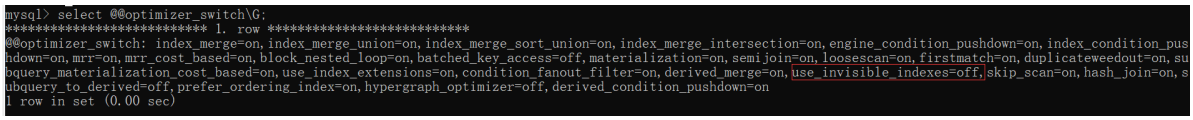
```
mysql> explain select * from t1 where i=1;
+----+-------------+-------+------------+------+---------------+-------+---------+-------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key   | key_len | ref   | rows | filtered | Extra |
+----+-------------+-------+------------+------+---------------+-------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | t1    | NULL       | ref  | i_idx         | i_idx | 5       | const |    1 |   100.00 | NULL  |
+----+-------------+-------+------------+------+---------------+-------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.00 sec)

mysql> explain select * from t1 where j=1;
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra       |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
|  1 | SIMPLE      | t1    | NULL       | ALL  | NULL          | NULL | NULL    | NULL |    1 |   100.00 | Using where |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
```

这里可以看到隐藏索引不会用上。

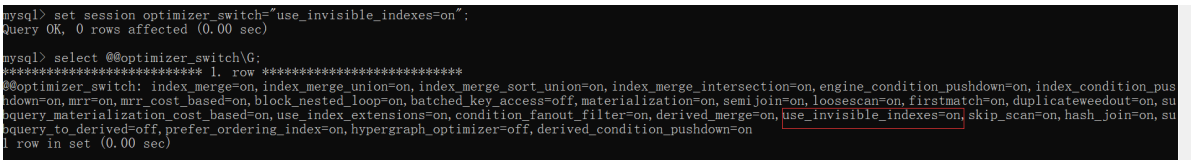
这里可以通过优化器的开关，打开一个设置，方便我们对隐藏索引进行设置。

```
select @@optimizer_switch\G;    --查看 各种参数
```



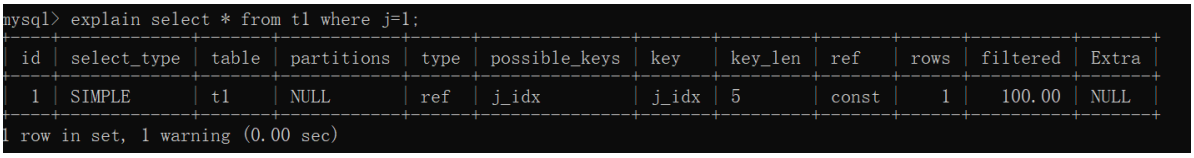
红色的部分就是默认查询优化器对隐藏索引不可见，我们可以通过参数进行修改。确保我们可以用隐藏索引进行测试。

```
set session optimizer_switch="use_invisible_indexes=on';    --在会话级别设置查询优化器可以看到隐藏索引
```



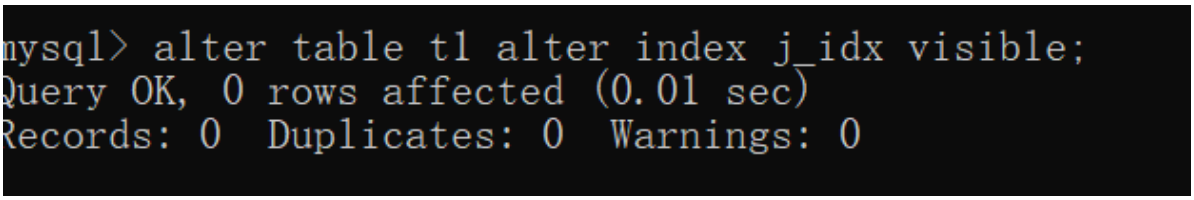
再使用查询优化器看下：

```
explain select * from t1 where j=1;
```



把隐藏索引变成可见索引（正常索引）

```
alter table t1 alter index j_idx visible;    --变成可见
alter table t1 alter index j_idx invisible;    --变成不可见(隐藏索引)
```



最后一点，不能把主键设置成不可见的索引（隐藏索引）（MySQL做了限制）

1.7.2.2. 降序索引

MySQL 8.0开始真正支持降序索引 (descendingindex) 。只有InnoDB存储引擎支持降序索引，只支持BTREE降序索引。另外MySQL8.0不再对GROUP BY操作进行隐式排序。

在MySQL中创建一个t2表

```
create table t2(c1 int,c2 int,index idx1(c1 asc,c2 desc));

show create table t2\G
```

```
mysql> show create table t2\G
*************************** 1. row ***************************
       Table: t2
Create Table: CREATE TABLE `t2` (
  `c1` int DEFAULT NULL,
  `c2` int DEFAULT NULL,
  KEY `idx1` (`c1`,`c2` DESC)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

如果是5.7中，则没有显示升序还是降序信息

```
mysql> create table t2(c1 int,c2 int,index idx1(c1 asc,c2 desc));
Query OK, 0 rows affected (0.02 sec)

mysql> show create table t2\G
*************************** 1. row ***************************
       Table: t2
Create Table: CREATE TABLE `t2` (
  `c1` int(11) DEFAULT NULL,
  `c2` int(11) DEFAULT NULL,
  KEY `idx1` (`c1`,`c2`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1
1 row in set (0.00 sec)
```

我们插入一些数据，给大家演示下降序索引的使用

```
insert into t2(c1,c2) values(1,100),(2,200),(3,150),(4,50);
```

```
mysql> select * from t2;
+------+------+
| c1   | c2   |
+------+------+
|    1 |  100 |
|    2 |  200 |
|    3 |  150 |
|    4 |   50 |
+------+------+
```

看下索引使用情况

```
explain select * from t2 order by c1,c2 desc;
```

```
mysql> explain select * from t2 order by c1,c2 desc;
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-------------+
| id | select_type | table | partitions | type  | possible_keys | key  | key_len | ref  | rows | filtered | Extra       |
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-------------+
|  1 | SIMPLE      | t2    | NULL       | index | NULL          | idx1 | 10      | NULL |    4 |   100.00 | Using index |
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-------------+
1 row in set, 1 warning (0.00 sec)
```

我们在5.7对比一下

```
mysql> explain select * from t2 order by c1,c2 desc;
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-----------------------------+
| id | select_type | table | partitions | type  | possible_keys | key  | key_len | ref  | rows | filtered | Extra                       |
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-----------------------------+
|  1 | SIMPLE      | t2    | NULL       | index | NULL          | idx1 | 10      | NULL |    4 |   100.00 | Using index; Using filesort |
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-----------------------------+
1 row in set, 1 warning (0.00 sec)
```

这里说明，这里需要一个额外的排序操作，才能把刚才的索引利用上。

我们把查询语句换一下

```
explain select * from t2 order by c1 desc,c2 ;
```

MySQL8中使用了

```
mysql> explain select * from t2 order by c1 desc,c2 ;
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-----------------------------------------+
| id | select_type | table | partitions | type  | possible_keys | key  | key_len | ref  | rows | filtered | Extra                                   |
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-----------------------------------------+
|  1 | SIMPLE      | t2    | NULL       | index | NULL          | idx1 | 10      | NULL |    4 |   100.00 | Backward index scan; Using index |
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-----------------------------------------+
1 row in set, 1 warning (0.00 sec)
```

另外还有一点，就是group by语句在 8之后不再默认排序

```
select count(*),c2 from t2 group by c2;
```

```
mysql> select count(*),c2 from t2 group by c2;
+----------+------+
| count(*) | c2   |
+----------+------+
|        1 |  100 |
|        1 |  200 |
|        1 |  150 |
|        1 |   50 |
+----------+------+
4 rows in set (0.00 sec)
```

MySQL8

```
mysql> select count(*),c2 from t2 group by c2;
+----------+------+
| count(*) | c2   |
+----------+------+
|        1 |   50 |
|        1 |  100 |
|        1 |  150 |
|        1 |  200 |
+----------+------+
```

MySQL5.7

在8要排序的话，就需要手动把排序语句加上

```
select count(*),c2 from t2 group by c2 order by c2;
```

```
mysql> select count(*),c2 from t2 group by c2 order by c2;
+----------+------+
| count(*) | c2   |
+----------+------+
|        1 |   50 |
|        1 |  100 |
|        1 |  150 |
|        1 |  200 |
+----------+------+
4 rows in set (0.00 sec)
```

1.7.2.3. 函数索引

之前我们知道，如果在查询中加入了函数，索引不生效，所以MySQL8引入了函数索引。

MySQL 8.0.13开始支持在索引中使用函数（表达式)的值。支持降序索引，支持JSON 数据的索引
函数索引基于虚拟列功能实现。

使用函数索引（表达式）

```
create table t3(c1 varchar(10),c2 varchar(10));
create index idx_c1 on t3(c1);   --普通索引
create index func_idx on t3( (UPPER(c2)) );   --一个大写的函数索引
```

```
mysql> create table t3(c1 varchar(10),c2 varchar(10));
Query OK, 0 rows affected (0.02 sec)

mysql> create index func_idx on t3( (UPPER(c2)) );
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> create index idx_c1 on t3(c1);
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
show index from t3\G
```

```
mysql> show index from t3\G
*************************** 1. row ***************************
        Table: t3
   Non_unique: 1
     Key_name: idx_c1
 Seq_in_index: 1
  Column_name: c1
    Collation: A
  Cardinality: 0
     Sub_part: NULL
       Packed: NULL
         Null: YES
   Index_type: BTREE
      Comment:
Index_comment:
      Visible: YES
   Expression: NULL
*************************** 2. row ***************************
        Table: t3
   Non_unique: 1
     Key_name: func_idx
 Seq_in_index: 1
  Column_name: NULL
    Collation: A
  Cardinality: 0
     Sub_part: NULL
       Packed: NULL
         Null: YES
   Index_type: BTREE
      Comment:
Index_comment:
      Visible: YES
   Expression: upper(`c2`)
2 rows in set (0.01 sec)

mysql>
```

普通索引

函数索引

```
explain select * from t3 where upper(c1)='ABC' ;
explain select * from t3 where upper(c2)='ABC' ;
```

```
mysql> explain select * from t3 where upper(c1)='ABC' ;
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra       |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
|  1 | SIMPLE      | t3    | NULL       | ALL  | NULL          | NULL | NULL    | NULL |    1 |   100.00 | Using where |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
1 row in set, 1 warning (0.00 sec)

mysql> explain select * from t3 where upper(c2)='ABC' ;
+----+-------------+-------+------------+------+---------------+----------+---------+-------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key      | key_len | ref   | rows | filtered | Extra |
+----+-------------+-------+------------+------+---------------+----------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | t3    | NULL       | ref  | func_idx      | func_idx | 43      | const |    1 |   100.00 | NULL  |
+----+-------------+-------+------------+------+---------------+----------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.00 sec)
```

使用函数索引（JSON）

```
create table t4(data json，index((CAST(data->>'$.name' as char(25)) )));
explain select * from t4 where CAST(data->>'$.name' as char(25)) = 'lijin ';
```

```
mysql> create table t4(data json,index((CAST(data->>'$.name'as char(25)))));
Query OK, 0 rows affected (0.02 sec)

mysql> explain select * from t4 where CAST(data->>'$.name' as char(25)) = 'lijin ';
+----+-------------+-------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys   | key             | key_len | ref   | rows | filtered | Extra |
+----+-------------+-------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | t4    | NULL       | ref  | functional_index | functional_index | 53      | const |    1 |   100.00 | NULL  |
+----+-------------+-------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.00 sec)
```

函数索引基于虚拟列功能实现

函数索引在MySQL中相当于新增了一个列，这个列会根据你的函数来进行计算结果，然后使用函数索引的时候就会用这个计算后的列作为索引。

1.7.3. 通用表表达式（CTE）

MySQL8.0开始支持通用表表达式(CTE）(common table expression)，即WITH子句。

简单入门:

以下SQL就是一个简单的CTE表达式，类似于递归调用，这段SQL中，首先执行select 1 然后得到查询结果后把这个值n送入 union all下面的 select n+1 from cte where n <10,然后一直这样递归调用union all下面sql语句。

```
WITH recursive cte(n) as
( select 1
  union ALL
  select n+1 from cte where n<10
)
select * from cte;
```

信息 结果1 概况 状态

n
1
2
3
4
5
6
7
8
9
10

案例介绍：

一个staff表，里面有id，有name还有一个 m_id，这个是对应的上级id。数据如下：

id	name	m_id
1	马老师	0
2	连老师	1
3	张老师	1
4	夏老师	1
5	李老师	4
6	严老师	2
7	晁老师	4
8	周老师	2
9	刘老师	3
10	邓老师	4

如果我们想查询出每一个员工的上下级关系，可以使用以下方式

递归CTE：

```
with recursive staff_view(id,name,m_id) as
(select id ,name ,cast(id as char(200))
 from staff where m_id =0
 union ALL
 select s2.id ,s2.name,concat(s1.m_id,'-',s2.id)
 from staff_view as s1 join  staff as s2
 on s1.id = s2.m_id
)
select * from staff_view order by id
```

信息 结果1 概况 状态

id	name	m_id
1	马老师	1
2	连老师	1-2
3	张老师	1-3
4	夏老师	1-4
5	李老师	1-4-5
6	严老师	1-2-6
7	晁老师	1-4-7
8	周老师	1-2-8
9	刘老师	1-3-9
10	邓老师	1-4-10

使用通用表表达式的好处就是上下级层级就算有4，5，6甚至更多层，都可以帮助我们遍历出来，而老的方式的写法SQL语句就要调整。

总结：

通用表表达式与派生表类似，就像语句级别的临时表或视图。CTE可以在查询中多次引用，可以引用其他CTE，可以递归。CTE支持SELECT/INSERT/UPDATE/DELETE等语句。

1.7.4. 窗口函数

MySQL 8.0支持窗口函数(Window Function)，也称分析函数。窗口函数与分组聚合函数类似，但是每一行数据都生成一个结果。聚合窗口函数: SUM /AVG / COUNT /MAX/MIN等等。

案例如下：sales表结构与数据如下：

year	country	product	sum
2021	USA	phote	1300
2022	USA	phote	1800
2020	USA	phote	2400
2021	CHA	phote	5300
2022	CHA	phote	7800
2020	CHA	phote	3400
2021	JPA	phote	3300
2022	JPA	phote	5800
2020	JPA	phote	6400
2021	USA	TV	3300
2022	USA	TV	800
2020	USA	TV	400
2021	CHA	TV	300
2022	CHA	TV	800
2020	CHA	TV	400
2021	JPA	TV	300
2022	JPA	TV	800
2020	JPA	TV	400

普通的分组、聚合（以国家统计）

```
SELECT country,sum(sum)
FROM sales
GROUP BY country
order BY country;
```

信息 结果1 概况 状态

country	sum(sum)
CHA	18000
JPA	17000
USA	10000

窗口函数（以国家汇总）

```
select year,country,product,sum,
    sum(sum) over (PARTITION by country) as country_sum
 from sales
order by country,year,product,sum;
```

year	country	product	sum	country_sum
2020	CHA	phote	3400	18000
2020	CHA	TV	400	18000
2021	CHA	phote	5300	18000
2021	CHA	TV	300	18000
2022	CHA	phote	7800	18000
2022	CHA	TV	800	18000
2020	JPA	phote	6400	17000
2020	JPA	TV	400	17000
2021	JPA	phote	3300	17000
2021	JPA	TV	300	17000
2022	JPA	phote	5800	17000
2022	JPA	TV	800	17000
2020	USA	phote	2400	10000
2020	USA	TV	400	10000
2021	USA	phote	1300	10000

窗口函数（计算平局值）

```
select year,country,product,sum,
            sum(sum) over (PARTITION by country) as country_sum,
            avg(sum) over (PARTITION by country) as country_avg
 from sales
order by country,year,product,sum;
```

year	country	product	sum	country_sum	country_avg
2020	CHA	phote	3400	18000	3000
2020	CHA	TV	400	18000	3000
2021	CHA	phote	5300	18000	3000
2021	CHA	TV	300	18000	3000
2022	CHA	phote	7800	18000	3000
2022	CHA	TV	800	18000	3000
2020	JPA	phote	6400	17000	2833.33333333
2020	JPA	TV	400	17000	2833.33333333
2021	JPA	phote	3300	17000	2833.33333333
2021	JPA	TV	300	17000	2833.33333333
2022	JPA	phote	5800	17000	2833.33333333
2022	JPA	TV	800	17000	2833.33333333
2020	USA	phote	2400	10000	1666.66666666
2020	USA	TV	400	10000	1666.66666666

专用窗口函数：

- 序号函数：ROW_NUMBER()、RANK()、DENSE_RANK()
- 分布函数：PERCENT_RANK()、CUME_DIST()
- 前后函数：LAG()、LEAD()
- 头尾函数：FIRST_VALUE()、LAST_VALUE()
- 其它函数：NTH_VALUE()、NTILE()

名称	参数	描述
ROW_NUMBER()	否	当前行在其分组内的序号。不管其排序结果中是否出现重复值，其排序结果都为：1,2,3,4,5。
DENSE_RANK()	否	不间断的组内排序。使用这个函数时，可以出现1,1,2,2这种形式的分组。
RANK()	否	间断的组内排序。其排序结果可能出现如下结果：1,1,3,4,4,6。
PERCENT_RANK()	否	累计百分比。该函数的计算结果为：**小于**该条记录值的所有记录的行数/该分组的总行数-1. 所以该记录的返回值为[0,1]。
CUME_DIST()	否	累计分布值。即分组值**小于等于**当前值的行数与分组总行数的比值。取值范围为(0,1]。
LAG()	是;lag(expr,[N,[default]])	从当前行开始往前取第N行，如果N缺失，默认为1。如果不存在前一行，则默认返回default。default默认值为NULL。
LEAD()	是;lead(expr,[N,[default]])	从当前行开始往后取第N行。函数功能与lag()相反，其余与lag()相同。
FIRST_VALUE()	是;first_value(expr)	返回分组内截止当前行的第一个值。
LAST_VALUE()	是;last_value(expr)	返回分组内截止当前行的最后一个值。
NTH_VALUE()	是;nth_value(expr,N)	返回分组内截止当前行的第N行。与first_value\last_value\nth_value函数功能相似，只是返回分组内截止当前行的不同行号的数据。
NTILE()	是;ntile(N)	返回当前行在分组内的分桶号。在计算时要先将该分组内的所有数据划分成N个桶，之后返回每个记录所在的分桶号。返回范围从1到N。

窗口函数（排名）

用于计算分类排名的排名窗口函数，以及获取指定位置数据的取值窗口函数

```
SELECT
    YEAR,
    country,
    product,
    sum,
    row_number() over (ORDER BY sum) AS 'rank',
        rank() over (ORDER BY sum) AS 'rank_1'
FROM
    sales;
```

信息 结果1 概况 状态

YEAR	country	product	sum	rank	rank_1
2021	CHA	TV	300	1	1
2021	JPA	TV	300	2	1
2020	USA	TV	400	3	3
2020	CHA	TV	400	4	3
2020	JPA	TV	400	5	3
2022	USA	TV	800	6	6
2022	CHA	TV	800	7	6
2022	JPA	TV	800	8	6
2021	USA	phote	1300	9	9

```
SELECT
    YEAR,
    country,
    product,
    sum,
    sum(sum) over (PARTITION by country order by sum rows unbounded preceding) as
sum_1
FROM
    sales order by country,sum;
```

```
SELECT
  YEAR,
  country,
  product,
  sum,
  sum(sum) over (PARTITION by country order by sum rows unbounded preceding) as sum_1
FROM
  sales order by country,sum;
```

累计总和

信息 结果1 概况 状态

YEAR	country	product	sum	sum_1
2021	CHA	TV	300	300
2020	CHA	TV	400	700
2022	CHA	TV	800	1500
2020	CHA	phote	3400	4900
2021	CHA	phote	5300	10200
2022	CHA	phote	7800	18000
2021	JPA	TV	300	300
2020	JPA	TV	400	700
2022	JPA	TV	800	1500
2021	JPA	phote	3300	4800
2022	JPA	phote	5800	10600

当然可以做的操作很多，具体见官网：

https://dev.mysql.com/doc/refman/8.0/en/window-function-descriptions.html

1.7.5. 原子DDL操作

MySQL 8.0 开始支持原子 DDL 操作，其中与表相关的原子 DDL 只支持 InnoDB 存储引擎。一个原子 DDL 操作内容包括：更新数据字典，存储引擎层的操作，在 binlog 中记录 DDL 操作。支持与表相关的 DDL：数据库、表空间、表、索引的 CREATE、ALTER、DROP 以及 TRUNCATE TABLE。支持的其他 DDL ：存储程序、触发器、视图、UDF 的 CREATE、DROP 以及ALTER 语句。支持账户管理相关的 DDL：用户和角色的 CREATE、ALTER、DROP 以及适用的 RENAME，以及 GRANT 和 REVOKE 语句。

```
drop table t1,t2;
```

上面这个语句，如果只有t1表，没有t2表。在MySQL5.7与8 的表现是不同的。

5.7会删除t1表。而在8中因为报错了，整个是一个原子操作，所以不会删除t1表。

1.7.6. JSON增强

具体看官网信息，英文好的直接看，英文不好的找个翻译工具即可看懂

MySQL :: MySQL 8.0 Reference Manual :: 11.5 The JSON Data Type

11.5 JSON 数据类型

- 创建 JSON 值
- JSON 值的规范化、合并和自动打包
- 搜索和修改 JSON 值
- JSON 路径语法
- JSON 值的比较和排序
- 在 JSON 和非 JSON 值之间进行转换
- JSON 值的聚合

1.7.7. InnoDB其他改进功能

自增列持久化

MySQL 5.7 以及早期版本，InnoDB 自增列计数器（AUTO_INCREMENT）的值只存储在内存中。MySQL 8.0 每次变化时将自增计数器的最大值写入 redo log，同时在每次检查点将其写入引擎私有的系统表。解决了长期以来的自增字段值可能重复的 bug。

死锁检查控制

MySQL 8.0 （MySQL 5.7.15）增加了一个新的动态变量，用于控制系统是否执行 InnoDB 死锁检查。对于高并发的系统，禁用死锁检查可能带来性能的提高。

```
innodb_deadlock_detect
```

锁定语句选项

SELECT ... FOR SHARE 和 SELECT ... FOR UPDATE 中支持 NOWAIT、SKIP LOCKED 选项。对于 NOWAIT，如果请求的行被其他事务锁定时，语句立即返回。对于 SKIP LOCKED，从返回的结果集中移除被锁定的行。

InnoDB 其他改进功能。

- 支持部分快速 DDL，ALTER TABLE ALGORITHM=INSTANT;
- InnoDB 临时表使用共享的临时表空间 ibtmp1。
- 新增静态变量 innodb_dedicated_server，自动配置 InnoDB 内存参数：innodb_buffer_pool_size/innodb_log_file_size 等。
- 默认创建 2 个 UNDO 表空间，不再使用系统表空间。
- 支持 ALTER TABLESPACE ... RENAME TO 重命名通用表空间。